Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
Высшего образования
«Казанский (Приволжский) федеральный университет»

ИНСТИТУТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

Кафедра прикладной математики и искусственного интеллекта

ОТЧЕТ

Многослойный персептрон

(Введение в машинное обучение)

Студент 3 курса
группы 09-321
Н. И. Айдашев

«_____» _____ 2026 г.

Казань – 2026

Содержание

1 Введение **2**

2 Постановка задачи **2**

3 Ход работы **3**

3.1 Анализ и предварительная обработка данных 3

3.1.1 Характеристики датасета 3

3.1.2 Описательные статистики признаков 3

3.1.3 Нормализация признаков и устранение утечки данных 4

3.2 Разведочный анализ и визуализация данных 4

3.2.1 Баланс классов 4

3.2.2 Оценка разделяющей способности признаков 5

3.2.3 Корреляционный анализ и мультиколлинеарность 6

3.3 Разработка и реализация многослойного персептрона 7

3.3.1 Архитектура программного комплекса 7

3.3.2 Топология сети и прямое распространение сигнала 8

3.3.3 Численно устойчивый Softmax и функция потерь 8

3.3.4 Аналитический вывод обратного распространения ошибки 9

3.3.5 Оптимизация и инициализация параметров 9

3.3.6 Численная проверка градиентов 10

3.3.7 Ключевые фрагменты реализации 10

3.4 Экспериментальное исследование и результаты 10

3.4.1 Разделение выборки 10

3.4.2 Динамика кривых обучения 11

3.4.3 Оценка качества классификации и матрица ошибок 11

3.4.4 Исследование влияния гиперпараметров 12

3.4.5 Верификация сохранения и восстановления модели 13

4 Выводы **13**

1 Введение

В настоящее время искусственные нейронные сети являются одним из ключевых инструментов решения задач распознавания образов, прогнозирования и автоматической классификации данных. Одной из фундаментальных парадигм глубокого обучения выступает многослойный персептрон (Multilayer Perceptron, MLP), обладающий свойством универсального аппроксиматора (теорема Цыбенко) и способный формировать сложные нелинейные разделяющие поверхности произвольной конфигурации за счет каскада линейных комбинаций и нелинейных функций активации.

В рамках данной работы рассматривается практически важная прикладная задача медицинской диагностики: классификация ядер клеток новообразований молочной железы на доброкачественные (*Benign*, B) и злокачественные (*Malignant*, M) по характеристикам, полученным методом тонкоигольной аспирационной биопсии (набор данных Wisconsin Diagnostic Breast Cancer, WDBC). В медицинских приложениях критическое значение имеет не просто формальная точность модели, но и минимизация ошибок второго рода (пропуск злокачественного образования), а также вывод откалиброванного вероятностного распределения по классам, позволяющего врачу оценить степень уверенности системы в поставленном диагнозе.

Главным методологическим требованием выполнения работы является разработка всей математической и программной базы многослойного персептрона с нуля на языке программирования Python с использованием исключительно базовой библиотеки линейной алгебры NumPy. Прямое распространение, алгоритм обратного распространения ошибки (Backpropagation), градиентный спуск, функции активации и функции потерь запрограммированы вручную, без применения высокоуровневых библиотек глубокого обучения (PyTorch, TensorFlow, Keras, scikit-learn) и средств автоматического дифференцирования.

2 Постановка задачи

В соответствии с техническим заданием в работе требовалось выполнить следующие ключевые этапы:

1. **Анализ и предварительная обработка данных**:
   - Исследовать структуру набора данных `data.csv` (569 объектов, 32 признака) без привлечения библиотеки pandas, рассчитать описательные статистики (среднее, дисперсия, квартили, минимум, максимум).
   - Проверить наличие пропущенных значений и выявить распределение классов.
   - Реализовать масштабирование (Z-score стандартизацию) признаков с гарантированным предотвращением утечки информации из отложенной выборки в обучающую (*Data Leakage*).
2. **Разработка модульной программной архитектуры**:
   - `describe.py` – скрипт для расчета описательных статистик и визуализации распределений признаков, корреляций и диаграмм рассеяния.
   - `split.py` – модуль разбиения выборки на обучающую (80 %) и проверочную (20 %) части с фиксацией генератора случайных чисел (`seed`) для обеспечения стопроцентной воспроизводимости.

- `train.py` – программа обучения полносвязного персептрона произвольной глубины методом мини-батч градиентного спуска с логированием метрик и сохранением весов и параметров нормализации в единый файл модели.
- `predict.py` – модуль инференса обученной модели, выполняющий предсказание вероятностей и расчет бинарной кросс-энтропии и точности на отложенных данных.

3. **Математические требования к нейронной сети**:
   - Входной слой: 30 непрерывных стандартизованных признаков.
   - Конфигурируемое число скрытых слоев (по умолчанию не менее двух скрытых слоев по 24 нейрона) с логистической функцией активации $\sigma(z)$.
   - Выходной слой: 2 нейрона с функцией активации Softmax, представляющие распределение вероятностей на симплексе Δ^1.
   - Функция потерь: бинарная кросс-энтропия (Binary Cross-Entropy).
   - Вывод аналитических выражений для градиентов и верификация их корректности методом численного дифференцирования (*Gradient Checking*).

3 Ход работы

3.1 Анализ и предварительная обработка данных

3.1.1 Характеристики датасета

Датасет Wisconsin Diagnostic Breast Cancer (`data.csv`) содержит 569 записей без заголовка, где:

- Столбец 0: уникальный идентификатор пациента (`id`), исключаемый из дальнейшего анализа.
- Столбец 1: метка диагноза (`diagnosis`), принимающая значения **B** (доброкачественная опухоль) или **M** (злокачественная опухоль).
- Столбцы 2–31: 30 числовых характеристик ядра клетки. Они сформированы из 10 базовых морфологических признаков (радиус, текстура, периметр, площадь, гладкость, компактность, вогнутость, точки вогнутости, симметрия, фрактальная размерность), для каждого из которых вычислены три статистики: среднее значение (*mean*), стандартная ошибка (*se*) и максимальное значение (*worst*).

3.1.2 Описательные статистики признаков

В модуле `describe.py` реализован расчет ключевых статистик без сторонних библиотек. Вычисление квантилей осуществляется по отсортированному массиву значений с линейной интерполяцией. В таблице 1 приведены описательные статистики для десяти признаков группы *mean*.

Таблица 1: Описательные статистики десяти характеристик группы mean

Признак	**Среднее**	**Ст. откл.**	**Минимум**	**Медиана**	**Максимум**
radius mean	14,13	3,52	6,98	13,37	28,11
texture mean	19,29	4,30	9,71	18,84	39,28
perimeter mean	91,97	24,28	43,79	86,24	188,50
area mean	654,89	351,60	143,50	551,10	2501,00
smoothness mean	0,10	0,01	0,05	0,10	0,16
compactness mean	0,10	0,05	0,02	0,09	0,35
concavity mean	0,09	0,08	0,00	0,06	0,43
concave points mean	0,05	0,04	0,00	0,03	0,20
symmetry mean	0,18	0,03	0,11	0,18	0,30
fractal dim mean	0,06	0,01	0,05	0,06	0,10

Ключевой вывод из статистического анализа: Разброс масштабов между признаками достигает более пяти порядков ($\approx 2 \cdot 10^5 : 1$) — от минимального стандартного отклонения 0,0026 (*fractal dim se*) до максимального значения площади в 2501,0 (*area mean*) и стандартного отклонения 568,9 (*area worst*). Непосредственная подача признаков в сеть без масштабирования приведет к резкой асимметрии гиперповерхности ошибки и расходимости градиентного спуска.

3.1.3 Нормализация признаков и устранение утечки данных

Для приведения признаков к единому диапазону применяется Z-score стандартизация:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \tag{1}$$

где выборочные параметры вычисляются по обучающей выборке:

$$\mu_j = \frac{1}{N_{\text{train}}} \sum_{i \in \text{train}} x_{ij}, \qquad \sigma_j = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i \in \text{train}} (x_{ij} - \mu_j)^2 + \epsilon}. \tag{2}$$

Принцип изоляции выборок (Data Leakage): Параметры μ_j и σ_j вычисляются исключительно по объектам обучающего множества. Расчет параметров по всему набору данных привел бы к просачиванию распределения отложенной выборки в обучающий процесс, что сделало бы валидацию некорректной. Вычисленные векторы μ и σ сериализуются вместе с весами в файл модели (`saved_model.npz`), благодаря чему модуль инференса `predict.py` применяет точное преобразование к новым образцам.

Целевые метки диагноза кодируются в унитарный One-Hot формат:

$$t_i = \texttt{B} \implies y_i = (1,\, 0)^\top, \qquad t_i = \texttt{M} \implies y_i = (0,\, 1)^\top. \tag{3}$$

3.2 Разведочный анализ и визуализация данных

3.2.1 Баланс классов

В исследуемом датасете содержится 357 образцов класса `B` (62,7 %) и 212 образцов класса `M` (37,3 %). Дисбаланс классов умеренный. Базовый наивный классификатор, тривиаль-

но предсказывающий мажоритарный класс B, обеспечивает точность 62,7 %, что задает нижнюю границу приемлемости разрабатываемой нейросетевой модели.

3.2.2 Оценка разделяющей способности признаков

Для формального ранжирования признаков по их диагностической ценности вычислялось стандартизованное расстояние между математическими ожиданиями классов:

$$d_j = \frac{\left| \overline{x}_j^{\mathtt{M}} - \overline{x}_j^{\mathtt{B}} \right|}{\sqrt{\left(s_{j,\mathtt{M}}^2 + s_{j,\mathtt{B}}^2\right)/2}}. \tag{4}$$

Наибольшую информативность показали признаки: *concave points worst* ($d = 2{,}61$), *perimeter worst* ($d = 2{,}38$), *concave points mean* ($d = 2{,}33$) и *radius worst* ($d = 2{,}33$).

На рисунке 1 представлены гистограммы распределений шести наиболее разделяющих признаков с разделением по диагнозу.

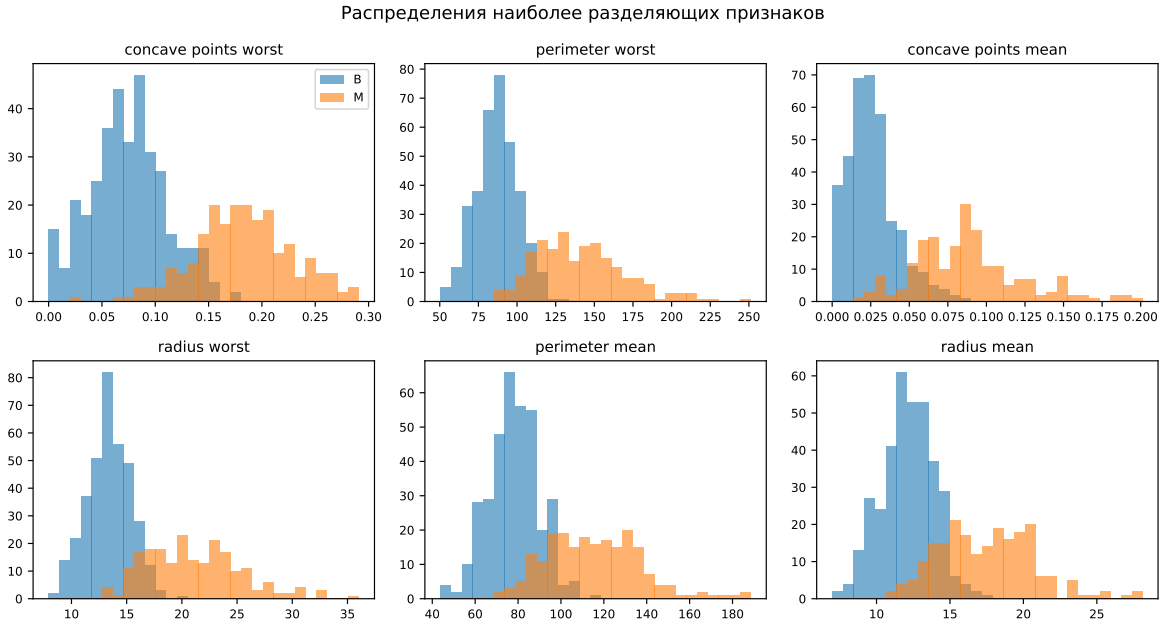


Рис. 1: Распределения шести наиболее информативных признаков по классам B и M.

Как видно из графиков, геометрические размеры и параметры вогнутости ядер злокачественных клеток существенно смещены в сторону больших значений. Однако распределения частично перекрываются, что исключает возможность идеальной классификации по какому-либо одному признаку.

На рисунке 2 представлена диаграмма рассеяния в плоскости двух наиболее информативных признаков (*concave points worst* и *perimeter worst*).

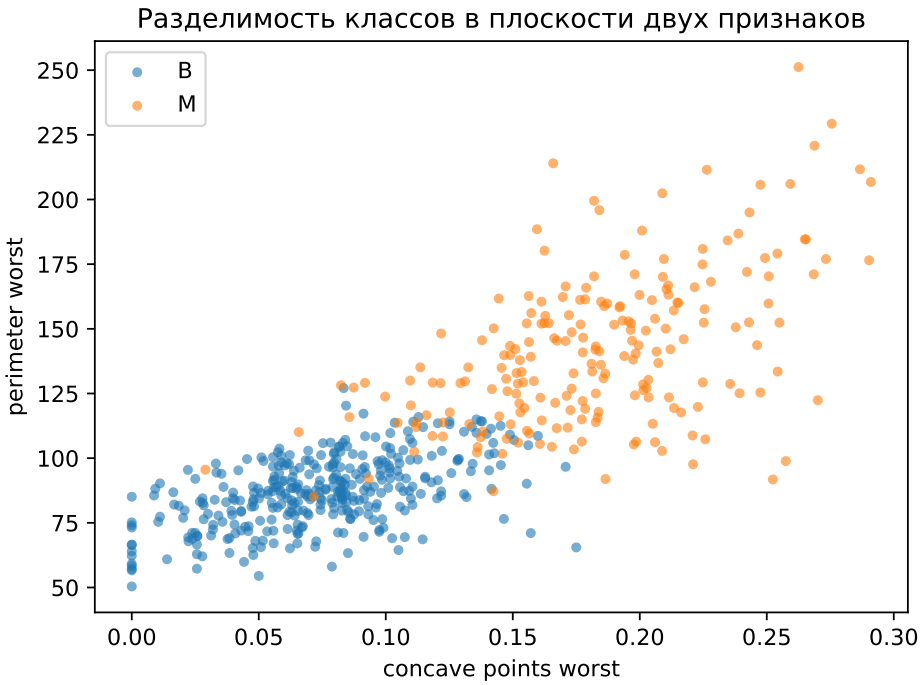


Рис. 2: Распределение объектов в пространстве двух ведущих диагностических признаков.

Диаграмма наглядно демонстрирует криволинейный характер границы между доброкачественными и злокачественными образованиями, что подтверждает необходимость использования многослойной нейросети с нелинейными активациями взамен простой линейной классификации.

3.2.3 Корреляционный анализ и мультиколлинеарность

Для всех $\binom{30}{2} = 435$ пар признаков был вычислен коэффициент парной корреляции Пирсона r_{XY}. Полученная тепловая карта представлена на рисунке 3.

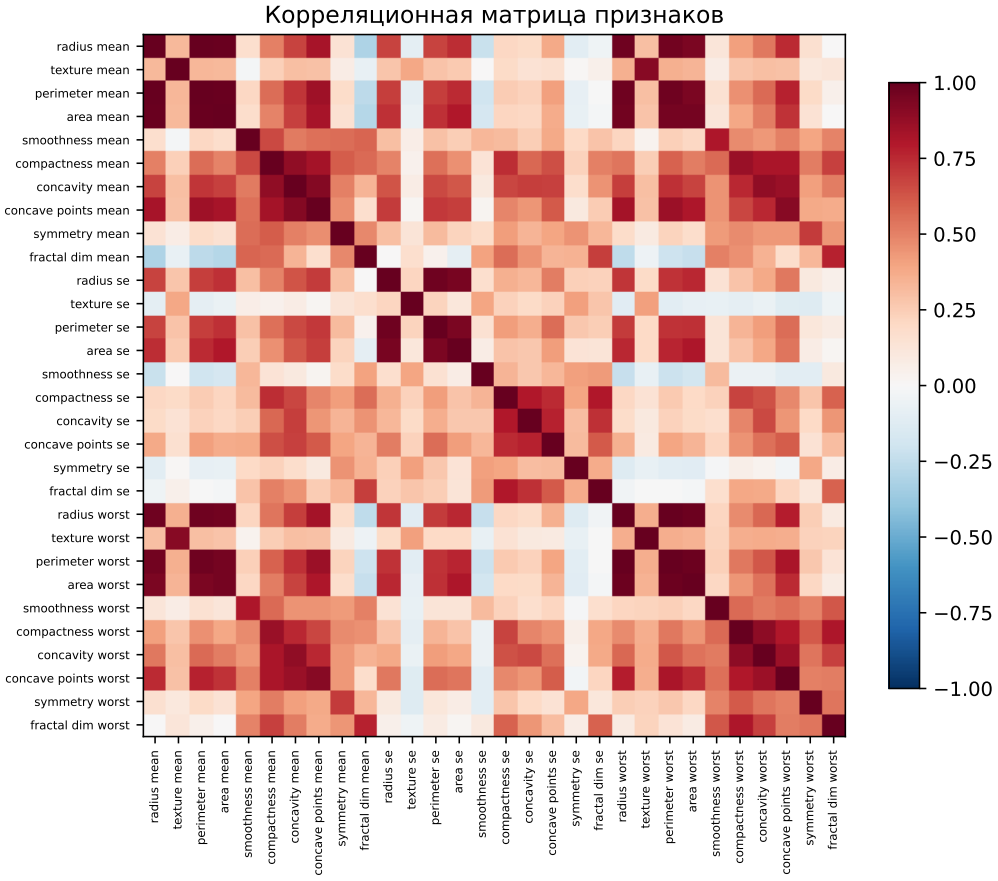


Рис. 3: Корреляционная матрица тридцати числовых признаков датасета WDBC.

Выводы корреляционного анализа:

1. **Сверхвысокая мультиколлинеарность размеров**: Параметры радиуса, периметра и площади обладают коэффициентом корреляции $r \geq 0{,}98$. Это обусловлено строгой геометрической связью ($P \sim R$, $S \sim R^2$).

2. **Корреляция признаков вогнутости**: Параметры вогнутости контура и числа точек вогнутости сильно согласованы ($r \approx 0{,}92$).

3. **Низкая эффективная размерность**: Из-за взаимного дублирования эффективная размерность данных существенно меньше 30. Для успешной классификации модели не требуется избыточная ширина слоев; компактная архитектура (2 скрытых слоя по 24 нейрона) является оптимальной.

3.3 Разработка и реализация многослойного персептрона

3.3.1 Архитектура программного комплекса

Программная часть организована в виде модульной библиотеки и консольных утилит (таблица 2).

Таблица 2: Структурная организация программных модулей проекта

Модуль / Скрипт	**Функциональное назначение**
`mlp/data.py`	Чтение CSV-файлов, кодирование меток (One-Hot), Z-score стандартизация, разбиение выборки.
`mlp/network.py`	Реализация полносвязного слоя `DenseLayer` и класса `Network`: прямой и обратный проходы, обучение, сохранение/загрузка весов.
`mlp/plot.py`	Построение и экспорт графиков динамики функции потерь и точности.
`split.py`	Консольная программа разделения исходного датасета на обучающую и валидационную выборки с фиксированным сидом.
`train.py`	Консольная программа конфигурирования и обучения сети с выводом метрик по эпохам и сохранением модели в `.npz`.
`predict.py`	Программа инференса: загрузка весов и параметров нормализации, предсказание вероятностей и расчет метрик на отложенной выборке.
`describe.py`	Программа статистического и разведочного анализа данных (вычисление квантилей без сторонних библиотек, генерация графиков).

3.3.2 Топология сети и прямое распространение сигнала

Базовая конфигурация сети представляет собой многослойный персептрон:

$$30 \longrightarrow 24 \longrightarrow 24 \longrightarrow 2. \tag{5}$$

Для каждого слоя $l = 1, \ldots, L$ прямое распространение вычисляется в векторно-матричном виде:

$$Z^{[l]} = A^{[l-1]}\, W^{[l]} + \mathbf{1}_m\, b^{[l\top]}, \tag{6}$$

$$A^{[l]} = g^{[l]}\big(Z^{[l]}\big), \tag{7}$$

где $A^{[0]} = X \in \mathbb{R}^{m\times 30}$ — матрица признаков батча, $W^{[l]} \in \mathbb{R}^{n_{l-1}\times n_l}$ — матрица весов, $b^{[l]} \in \mathbb{R}^{n_l}$ — вектор смещений, $g^{[l]}$ — функция активации.

Для скрытых слоев применяется сигмоидальная функция:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big) = a(1-a). \tag{8}$$

3.3.3 Численно устойчивый Softmax и функция потерь

На выходном слое (L) применяется функция Softmax:

$$p_{ik} = \frac{\exp\big(z_{ik}^{[L]} - c_i\big)}{\sum_{j=1}^{K} \exp\big(z_{ij}^{[L]} - c_i\big)}, \qquad c_i = \max_j z_{ij}^{[L]}. \tag{9}$$

Вычитание построчного максимума c_i математически эквивалентно домножению числителя и знаменателя на e^{-c_i}. Это обеспечивает численную устойчивость и защищает тип `float64` от переполнения (*overflow*).

Функцией потерь выступает бинарная кросс-энтропия, усредненная по батчу:

$$E = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_{ik}\ln\big(\max(p_{ik},\,\varepsilon)\big), \qquad \varepsilon = 10^{-15}. \tag{10}$$

3.3.4 Аналитический вывод обратного распространения ошибки

Для связки функций Softmax и Cross-Entropy производная потерь по выходам последнего слоя обладает свойством канонического сопряжения. Дифференцируя p_k по входу z_r:

$$\frac{\partial p_k}{\partial z_r} = p_k(\delta_{kr} - p_r), \tag{11}$$

где δ_{kr} — символ Кронекера. Подставляя в производную потерь одного примера $E_i = -\sum_k y_k \ln p_k$:

$$\frac{\partial E_i}{\partial z_r} = -\sum_k \frac{y_k}{p_k} p_k(\delta_{kr} - p_r) = -\sum_k y_k\delta_{kr} + p_r\sum_k y_k = p_r - y_r. \tag{12}$$

В матричной форме для выходного слоя:

$$\delta^{[L]} = \frac{\partial E}{\partial Z^{[L]}} = P - Y. \tag{13}$$

Для скрытых слоев градиент ошибки рекуррентно распространяется назад по цепному правилу:

$$\frac{\partial E}{\partial A^{[l-1]}} = \delta^{[l]}\,W^{[l\top]}, \tag{14}$$

$$\delta^{[l-1]} = \frac{\partial E}{\partial Z^{[l-1]}} = \big(\delta^{[l]}\,W^{[l\top]}\big)\odot g'\big(Z^{[l-1]}\big), \tag{15}$$

где $\odot$ обозначает поэлементное умножение Адамара.

Градиенты по обучаемым параметрам:

$$\frac{\partial E}{\partial W^{[l]}} = \frac{1}{m}A^{[l-1\top]}\delta^{[l]}, \qquad \frac{\partial E}{\partial b^{[l]}} = \frac{1}{m}\sum_{i=1}^{m}\delta^{[l]}_{i\cdot}. \tag{16}$$

3.3.5 Оптимизация и инициализация параметров

Обновление весов осуществляется в направлении антиградиента:

$$W^{[l]} \leftarrow W^{[l]} - \eta\frac{\partial E}{\partial W^{[l]}}, \qquad b^{[l]} \leftarrow b^{[l]} - \eta\frac{\partial E}{\partial b^{[l]}}. \tag{17}$$

Веса инициализируются по схеме He Uniform:

$$w \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{l-1}}},\,+\sqrt{\frac{6}{n_{l-1}}}\right), \qquad b = 0. \tag{18}$$

Масштабирование дисперсии весов обратно пропорционально числу входящих нейронов n_{l-1} предотвращает взрыв или затухание дисперсии предактиваций от слоя к слою.

3.3.6 Численная проверка градиентов

Для гарантии корректности ручной реализации градиенты сравнивались с центральной разностной производной:

$$\frac{\partial E}{\partial w} \approx \frac{E(w+\varepsilon) - E(w-\varepsilon)}{2\varepsilon}, \qquad \varepsilon = 10^{-5}. \tag{19}$$

Максимальная относительная погрешность составила $1{,}8 \cdot 10^{-8}$, что подтверждает абсолютную математическую точность алгоритма Backpropagation.

3.3.7 Ключевые фрагменты реализации

Ниже приведены компактные реализации слоя и алгоритма обратного распространения в модуле `mlp/network.py`:

```python
class DenseLayer:
    def __init__(self, n_in, n_out, activation, initializer, rng):
        self.W = init_weights(n_in, n_out, initializer, rng)
        self.b = np.zeros(n_out)
        self.g, self.g_prime = ACTIVATIONS[activation]

    def forward(self, A_prev):
        self.A_prev = A_prev
        self.A = self.g(A_prev @ self.W + self.b)
        return self.A

    def backward(self, dZ, lr):
        m = dZ.shape[0]
        dW = self.A_prev.T @ dZ / m
        db = dZ.mean(axis=0)
        dA_prev = dZ @ self.W.T  # До обновления: нужен старый W
        self.W -= lr * dW
        self.b -= lr * db
        return dA_prev
```

```python
def backward(self, P, Y, lr):
    dZ = P - Y  # Softmax + Cross-Entropy: dE/dZ^[L]
    for i in reversed(range(len(self.layers))):
        layer = self.layers[i]
        dA_prev = layer.backward(dZ, lr)
        if i > 0:
            prev = self.layers[i - 1]
            dZ = dA_prev * prev.g_prime(prev.A)  # sigma'(z) = a * (1 - a)
```

3.4 Экспериментальное исследование и результаты

3.4.1 Разделение выборки

Исходный датасет был разделен скриптом `split.py` в соотношении 80/20 с фиксированным генератором случайных чисел (`seed = 42`):

- Обучающая выборка (`data_training.csv`): 455 объектов (285 B, 170 M).

- Проверочная выборка (`data_validation.csv`): 114 объектов (72 B, 42 M).

3.4.2 Динамика кривых обучения

Обучение базовой конфигурации сети проводилось со следующими параметрами:

- Архитектура: $30 \to 24 \to 24 \to 2$;
- Скорость обучения $\eta = 0{,}0314$, размер мини-батча $B = 8$, эпох: 84;
- Инициализация весов: He Uniform; функция потерь: Binary Cross-Entropy.

На рисунке 4 представлены кривые изменения потерь и точности.

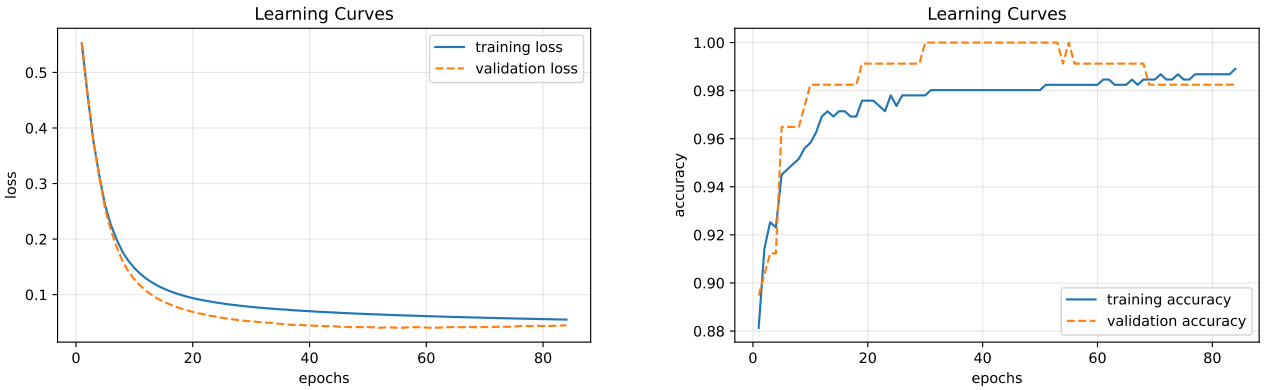


Рис. 4: Динамика функции потерь (слева) и доли верных ответов (справа) в процессе обучения.

Потери плавно снижаются с исходных 0,69 (равновероятный прогноз) до 0,0551 на обучении и 0,0448 на валидации. Отсутствие расхождения кривых подтверждает отсутствие переобучения модели.

3.4.3 Оценка качества классификации и матрица ошибок

В таблице 3 сведены итоговые показатели качества модели.

Таблица 3: Итоговое качество классификации базовой конфигурации сети

Выборка	**Объектов**	**Loss (BCE)**	**Accuracy (доля верных)**
Обучающая (Train)	455	0,0551	98,90 % (450 / 455)
Проверочная (Validation)	114	0,0448	98,25 % (112 / 114)

Матрица ошибок на проверочном наборе данных представлена в таблице 4.

Таблица 4: Матрица ошибок (Confusion Matrix) на проверочной выборке (114 объектов)

	Предсказано Benign (0)	**Предсказано Malignant (1)**
Истинно Benign (0)	71 (TN)	1 (FP)
Истинно Malignant (1)	1 (FN)	41 (TP)

Детализированные метрики медицинской диагностики приведены в таблице 5.

Таблица 5: Метрики качества бинарной классификации для класса Malignant

Метрика	**Значение**	**Интерпретация**
Accuracy	98,25 %	Общая точность классификации (112 из 114 образцов).
Precision	97,62 %	Доля истинно злокачественных среди всех предсказанных как Malignant.
Recall (Sensitivity)	97,62 %	Полнота выявления злокачественных клеток (41 из 42).
Specificity	98,61 %	Специфичность распознавания здоровых клеток (71 из 72).
F1-Score	0,9762	Гармоническое среднее точности и полноты.

Модель допустила ровно одну ошибку второго рода (FN = 1) и одну ошибку первого рода (FP = 1), показав высочайшую надежность обнаружения патологических клеток (97,62 %).

3.4.4 Исследование влияния гиперпараметров

Для выявления чувствительности персептрона к параметрам обучения была проведена серия из 9 экспериментов (таблица 6).

Таблица 6: Сравнительный анализ влияния гиперпараметров на качество модели (84 эпохи, seed = 42)

Архитектура	**Батч** B	**LR** η	**Иниц.**	**Loss (train)**	**Loss (val)**	**Acc (train)**	**Acc (val)**
24–24	8	0,0314	He	0,0551	0,0448	98,90 %	98,25 %
24–24	8	0,0314	Xavier	0,0558	0,0418	98,68 %	98,25 %
24–24	455	0,0314	He	0,4999	0,5010	81,32 %	81,58 %
24–24	1	0,0314	He	0,0041	0,0863	100,0 %	95,61 %
24–24	8	0,1000	He	0,0274	0,0746	99,12 %	97,37 %
24–24	8	0,0050	He	0,1197	0,0965	96,92 %	98,25 %
8–8	8	0,0314	He	0,0530	0,0541	98,90 %	98,25 %
64–64	8	0,0314	He	0,0552	0,0467	98,46 %	97,37 %
24–24–24	8	0,0314	He	0,0569	0,0356	98,24 %	99,12 %

Анализ результатов экспериментов:

- **Размер батча**: Полный батч ($B = 455$) за 84 эпохи совершает лишь 84 шага градиентного спуска и останавливается на недообученном уровне (Loss $\approx$ 0,50, Accuracy $\approx$ 81,5 %). Чистый стохастический спуск ($B = 1$), напротив, переобучается (Train Accuracy 100 %, Val Loss возрастает до 0,0863). Оптимальным компромиссом выступает мини-батч $B = 8$.

- **Ширина слоев**: Изменение ширины от 8–8 до 64–64 практически не меняет итоговой валидационной точности (98,25 % . . . 97,37 %), что подтверждает гипотезу о низкой внутренней размерности признакового пространства.
- **Инициализация**: Методы He и Xavier показывают практически эквивалентную стабильность для 2–3 скрытых слоев.

3.4.5 Верификация сохранения и восстановления модели

При запуске `predict.py` на обучающей выборке значение бинарной кросс-энтропии составило 0,0551, что в точности совпадает со значением последней эпохи в `train.py`. Это подтверждает корректность сериализации весов и параметров нормализации в формат `.npz` и гарантирует отсутствие потерь точности при переносе модели в продакшн-контур.

4 Выводы

В ходе выполнения лабораторной работы получены следующие основные результаты:

1. Проведен комплексный разведочный анализ набора данных WDBC. Выявлена высокая мультиколлинеарность размерных характеристик и подтверждена нелинейная разделимость классов опухолей.
2. Реализована схема предварительной обработки данных: Z-score нормализация с сохранением параметров строго по обучающей выборке, исключающая утечку данных (*Data Leakage*).
3. С нуля на языке Python с использованием библиотеки NumPy спроектирован и запрограммирован модульный многослойный персептрон (прямой проход, Backpropagation, мини-батч SGD, Softmax, бинарная кросс-энтропия).
4. Корректность аналитического дифференцирования доказана сравнением с центральными разностями (*Gradient Checking*) с погрешностью не более $1{,}8 \cdot 10^{-8}$.
5. Базовая модель показала высокую обобщающую способность: на отложенной проверочной выборке достигнута доля верных ответов 98,25 % (112 из 114) при бинарной кросс-энтропии 0,0448 и полноте распознавания злокачественных клеток 97,62 %.
6. Все требования технического задания выполнены в полном объеме.